

GitHub Copilot Cost Optimization Guide

1. Prompt Compression

- Keep prompts concise and specific.
- Avoid repeating context or instructions.
- Use clear requirements instead of lengthy descriptions.

2. Context Management

- Start with the smallest relevant context.
- Escalate from file → module → repository only when needed.
- Avoid sending entire codebases for simple tasks.

3. Output Control

- Use Copilot Instructions to define response format and coding standards.
- Request summaries or focused outputs instead of lengthy explanations.
- Reduce unnecessary output tokens.

4. Use the Right Workflow Mode

- **Ask** → Quick questions and code explanations
- **Plan** → Design discussions and feature planning
- **Agent** → Complex multi-step automation

Use the simplest mode that can accomplish the task.

5. Optimize Agent Configurations

- Create purpose-built Custom Agents.
- Use Skills and Subagents for reusable expertise.
- Reduce retries and improve consistency.

6. Model Routing & Reasoning Budget

- Use lightweight models for documentation, refactoring, and code generation.
- Reserve premium reasoning models for debugging, architecture, and migrations.
- Match model capability to task complexity.

7. MCP Tool Pruning

- Expose only the tools agents actually need.
- Remove unused MCP integrations.
- Reduce reasoning overhead and token consumption.

8. Enterprise Guardrails

- Configure Enterprise, Cost Center, and User budgets.
- Set alerts at 75%, 90%, and 100%.
- Review usage regularly and optimize before increasing budgets.

Key Takeaway

Don't focus on using fewer AI credits. Focus on getting more value from every AI credit consumed.